

## Playwright full questions

### Difference b/w playwright selenium and Cyperus?

|                                                                                                              |                              |                                    |                          |
|--------------------------------------------------------------------------------------------------------------|------------------------------|------------------------------------|--------------------------|
| <b>pose</b>                                                                                                  | Modern end-to-end testing    | General browser automation         | Lightweight testing tool |
|  <b>Speed</b>               | Fast (auto-waiting built-in) | Slower (manual waits often needed) | Fast for small projects  |
|  <b>Browser Support</b>     | Chromium, Firefox, WebKit    | All major browsers                 | Limited browser support  |
|  <b>Mobile Testing</b>      | Yes (emulates devices)       | Limited                            | Basic or none            |
|  <b>Smart Waiting</b>     | Yes (less flaky tests)       | No (needs manual waits)            | Yes                      |
|  <b>Setup</b>             | Easy with Node.js            | More setup (depends on language)   | Very simple              |
|  <b>Screenshots/Video</b> | Built-in                     | Needs extra setup                  | Basic support            |
|  <b>Languages</b>         | JavaScript/TypeScript        | Many (Java, Python, JS, etc.)      | Mostly JavaScript        |
|  <b>Community</b>         | Growing fast                 | Huge and mature                    | Small                    |

### 2) what are testing types supported by playwright

Playwright supports several types of testing to help ensure your web application works correctly. Here's a **simple breakdown** of the main testing types Playwright supports:

---

### ✓ 1. End-to-End (E2E) Testing

- Tests the full flow of your app from start to finish.
  - Example: Login → Navigate → Submit form → Logout.
- 

### ✓ 2. UI Testing

- Checks if buttons, forms, links, etc. are visible and working.
  - Example: Is the "Submit" button clickable? Is the error message shown?
- 

### ✓ 3. Functional Testing

- Verifies that specific features work as expected.
  - Example: Does the search feature return correct results?
- 

### ✓ 4. Visual Testing (Screenshots & Videos)

- Captures screenshots or videos to check layout and design.
  - Useful for detecting UI changes or bugs.
- 

### ✓ 5. API Testing (via request object)

- You can test REST APIs directly using Playwright.
  - Example: Send a GET request and check the response.
- 

### ✓ 6. Cross-Browser Testing

- Run tests on Chromium, Firefox, and WebKit.
  - Ensures your app works on all major browsers.
- 

### ✓ 7. Mobile Emulation Testing

- Simulates mobile devices and screen sizes.
  - Example: Test how your app looks on iPhone or Android.
- 

## ✓ 8. Accessibility Testing (basic)

- You can check for accessibility attributes like aria-label, roles, etc.
- Not full a11y testing, but helpful for basic checks.

### 3) How to install nodejs and npm in playwright?

To install Node.js and npm, I go to the official Node.js website (nodejs.org), download the LTS version, and run the installer. It automatically installs both Node.js and npm. After installation, I verify it by running `node -v` and `npm -v` in the terminal to check the versions.

### 4) Install playwright via npm?

```
npm init playwright@latest
```

to run the testcases-> **`npx playwright test Reethika.spec.ts --headed`**

to run in only one browser-> **`npx playwright test Reethika.spec.ts --project=chromium`**

### 5) Overview of Configuration file and its Test Configuration in playwright?

In Playwright, the configuration file is usually named `playwright.config.ts` or `.js`. It defines global settings for the test project like test directory, timeout, retries, browser options, and reporters. It helps standardize how tests run across environments. For example, we can set retries to handle flaky tests, use to define browser settings like headless mode, and projects to run tests across multiple browsers. This file makes Playwright flexible and easy to integrate with CI/CD pipelines.

### 6) Playwright test runner?

"Playwright Test Runner uses a config file (`playwright.config.ts`) to define global settings like test directory, timeouts, retries, and browser options. Tests are written using `test()` blocks and can run across multiple browsers in parallel. It supports features like fixtures, auto-waiting, and built-in reporters, making it ideal for modern web testing."

### 7) Record and Playback - Codegen

Playwright provides a feature called Codegen for record and playback. It automatically generates test scripts by recording user interactions with a browser. I can start it using the command `npx playwright codegen <URL>`. As I interact with the page, Playwright records actions like clicks, typing, and navigation, and generates code in real-time. This

helps quickly create test scripts without writing them manually. It's useful for learning selectors and speeding up test creation."

Command-> **npx playwright codegen https://example.com**

**8)Command Line execution with Playwright Test**

| Command                                   | Purpose                    |
|-------------------------------------------|----------------------------|
| npx playwright test                       | Run all tests              |
| npx playwright test tests/example.spec.ts | Run a specific test file   |
| npx playwright test --headed              | Run tests with browser UI  |
| npx playwright test --project=chromium    | Run tests in Chromium only |
| npx playwright test --reporter=html       | Generate HTML report       |

**9) Understanding the importance of Synchronization / Auto waiting**

Synchronization or auto-waiting in Playwright is crucial because it helps reduce test flakiness. Playwright automatically waits for elements to be ready before performing actions like clicks or typing. This means I don't need to manually add waits or sleep statements. For example, if a button appears after a delay, Playwright waits for it to be visible and enabled before clicking. This built-in smart waiting makes tests more stable and reliable, especially in dynamic web applications.

**10) Head and Headless mode browser automation**

In Playwright, browser automation can run in two modes: Headed and Headless. Headed mode runs with the browser UI visible, which is useful for debugging and visually observing the test flow. Headless mode runs without the browser UI, making it faster and ideal for CI/CD pipelines. By default, Playwright runs in headless mode, but we can switch to headed mode using the --headed flag or by setting headless: false in the configuration.

```
use: {  
  
  headless: false, // Runs in headed mode  
  
}  
  
npx playwright test --headed
```



## 11) Opening and closing browser instances

"In Playwright, browser instances are managed using the browser, context, and page objects. To open a browser, we use `playwright.chromium.launch()` or similar methods for other browsers. We can choose between headless or headed mode. After launching, we create a browser context and then a page to interact with. Once the test is complete, we close the page, context, and browser using `.close()` methods to free up resources. This helps ensure clean test execution and avoids memory leaks.

## 12) Browser contexts and Multi-user context, Multi pages

### Browser Contexts in Playwright

"A browser context in Playwright is like a separate browser session. It allows us to isolate cookies, local storage, and cache between tests. This is useful for testing multiple users or scenarios without interference. We create a context using `browser.newContext()` and then open pages within that context."

---

#### ✓ Multi-user Contexts

"Playwright supports multi-user testing by creating multiple browser contexts. Each context can simulate a different user with separate login sessions. This is useful for testing interactions between users, like messaging or admin-user workflows."

```
1  const browser = await chromium.launch();
2  const user1Context = await browser.newContext();
3  const user2Context = await browser.newContext();
4
5  const user1Page = await user1Context.newPage();
6  const user2Page = await user2Context.newPage();
7
8  await user1Page.goto('https://app.com/login');
9  await user2Page.goto('https://app.com/login');
10
11  // Simulate login for both users
12  await user1Page.fill('#username', 'user1');
```

---

## ✓ Multi-page Testing

"We can open multiple pages within the same context using `context.newPage()`. This helps test scenarios like popups, tabs, or simultaneous actions. Each page runs independently but shares the same session data if in the same context."

Handling multiple browser instances

```
import { chromium } from 'playwright';

(async () => {

  const browser1 = await chromium.launch();
  const browser2 = await chromium.launch();
  const context1 = await browser1.newContext();
  const context2 = await browser2.newContext();
  const page1 = await context1.newPage();
  const page2 = await context2.newPage();
  await page1.goto('https://example.com/user1');
  await page2.goto('https://example.com/user2');
  await browser1.close();
  await browser2.close();

})();
```

### 13) Browser Initiation from Config file and Browser Independent Initiation

In Playwright, we can define browser settings in the `playwright.config.ts` file using the `projects` array. This allows us to run tests across multiple browsers like Chromium, Firefox, and WebKit without changing the test code. Each project in the config represents a browser with its own settings. When we run `npx playwright test`, Playwright automatically launches the browsers defined in the config.

```
projects: [
  { name: 'Chromium', use: { browserName: 'chromium' } },
  { name: 'Firefox', use: { browserName: 'firefox' } },
  { name: 'WebKit', use: { browserName: 'webkit' } },
]
```

## Browser Independent Initiation (Manual Launch)

"Apart from using the config file, we can manually launch browsers in Playwright using code. This gives us more control over browser behavior, especially for custom scenarios. We use `playwright.chromium.launch()` or similar methods to start the browser, then create contexts and pages manually."

### Handling new / Child window

```
import { chromium } from 'playwright';

(async () => {
  const browser = await chromium.launch({ headless: false });
  const context = await browser.newContext();
  const page = await context.newPage();
  await page.goto('https://example.com');
  await browser.close();
})();
```

### 14) Handling new / Child window

In Playwright, when a new or child window opens—like a popup or a new tab—we can handle it using the `page.context().waitForEvent('page')` method. This allows us to capture and interact with the new page instance. It's useful for testing scenarios like login popups, payment gateways, or external links. Once we get the new page object, we can perform actions or assertions just like on the main page.

```
1 const [newPage] = await Promise.all([
2   context.waitForEvent('page'), // Wait for the new page
3   page.click('a[target="_blank"]') // Action that opens the new window
4 ]);
5
6 await newPage.waitForLoadState();
7 console.log(await newPage.title());
8
```

### 15) Handling new and tabs

In Playwright, when a new window or tab opens—like from a link with `target="_blank"`—we can handle it using `context.waitForEvent('page')`. This lets us capture the new page object and interact with it just like the main page. It's useful for testing popups, external links, or multi-tab workflows.

```

1  const [newPage] = await Promise.all([
2    context.waitForEvent('page'),      // Wait for the new tab/window
3    page.click('a[target="_blank"]'),  // Action that opens it
4  ]);
5
6  await newPage.waitForLoadState();
7  console.log(await newPage.title());
8

```

## 16) Browser Navigation Controls

In Playwright, browser navigation controls allow us to move between pages just like a user would. We use methods like `page.goto()` to navigate to a URL, `page.goBack()` to go to the previous page, and `page.goForward()` to move ahead. These are useful for testing navigation flows, history handling, and multi-step processes in web applications.

```

1  await page.goto('https://example.com');    // Navigate to a page
2  await page.click('#nextPage');             // Click a link
3  await page.goBack();                       // Go back to previous page
4  await page.goForward();                   // Go forward again
5

```

## 17) How to listen for navigation events (e.g., load, domcontentloaded)

In Playwright, we can listen for navigation events like `load` and `domcontentloaded` using the `page.waitForLoadState()` method. This helps ensure that the page has reached a certain loading stage before performing actions. It improves test stability by waiting for the right moment to interact with the page.

📌 'load' – when the load event is fired (full page load).

📌 'domcontentloaded' – when the DOM is ready (no need to wait for images).

📌 'networkidle' – when there are no network connections for at least 500 ms.

```

1  await page.goto('https://example.com');
2
3  // Wait for full page load (includes images, scripts, etc.)
4  await page.waitForLoadState('load');
5
6  // Or wait for DOM content to be loaded (faster, excludes images)
7  await page.waitForLoadState('domcontentloaded');
8

```

## 18) Why are selectors important for locating web elements?

"Selectors are important in Playwright because they help identify and locate specific elements on a web page to interact with them—like clicking a button, filling a form, or verifying text. A good selector ensures the test is stable and reliable. Playwright supports various types of selectors like CSS, text, XPath, and even role-based selectors. Choosing the right selector reduces flakiness and makes the test easier to maintain."

- Selectors are used to **locate elements** for actions and assertions.
- Playwright supports **CSS, XPath, text, role, and custom selectors**.
- **Stable selectors = less flaky tests.**
- Good selectors improve **test readability and maintainability**.

19) Playwright Inspector and Locating Strategies

Playwright Inspector

"Playwright Inspector is a debugging tool that helps visualize and step through test execution. It shows the browser, highlights elements being interacted with, and displays the corresponding code. We can launch it using the PWDEBUG=1 environment variable. It's very useful for identifying issues, understanding test flow, and refining selectors."

PWDEBUG=1 npx playwright test

Locating Strategies in Playwright

"Playwright supports multiple strategies to locate elements on a page. These include CSS selectors, text selectors, XPath, role-based selectors, and custom attributes. Choosing the right strategy improves test stability and readability."

🔍 Common Selector Types:

| Strategy      | Example                                                   | Use Case                          |
|---------------|-----------------------------------------------------------|-----------------------------------|
| CSS Selector  | <code>page.locator('button.submit')</code>                | Precise and fast                  |
| Text Selector | <code>page.getByText('Login')</code>                      | When text is unique               |
| Role Selector | <code>page.getByRole('button', { name: 'Submit' })</code> | Accessible and readable           |
| XPath         | <code>page.locator('//button[text()='OK']')</code>        | When CSS is not enough            |
| Test ID       | <code>page.getByTestId('login-button')</code>             | Best for stable, custom targeting |

## 20) Different Ways to Locate DOM Elements in Playwright

"Playwright provides multiple strategies to locate DOM elements, which helps in writing stable and readable tests. Choosing the right selector improves test reliability and reduces flakiness.

| Strategy       | Example                                                  | Use Case                          |
|----------------|----------------------------------------------------------|-----------------------------------|
| CSS Selector   | <code>page.locator('button.login')</code>                | Precise and fast                  |
| Text Selector  | <code>page.getByText('Submit')</code>                    | When text is unique               |
| Role Selector  | <code>page.getByRole('button', { name: 'Login' })</code> | Accessible and readable           |
| XPath          | <code>page.locator('//input[@id="email"]')</code>        | When CSS is not enough            |
| Test ID        | <code>page.getByTestId('username-input')</code>          | Best for stable, custom targeting |
| Label Selector | <code>page.getByLabel('Email')</code>                    | Useful for form fields            |

## 21) Handling Shadow DOM in Playwright

"Shadow DOM is used in modern web apps to encapsulate elements. Playwright can pierce through shadow DOM using standard locators, but sometimes we need to use `locator.evaluate()` or `locator.locator()` to go deeper."

```
1 // Accessing an element inside shadow DOM
2 const shadowHost = page.locator('custom-element');
3 const shadowButton = shadowHost.locator('button#inside-shadow');
4 await shadowButton.click();
5
```

Or using `evaluateHandle`:

```
1 const shadowRoot = await page.locator('custom-element').evaluateHandle(el => el.shadowRoot);
2 const button = await shadowRoot.asElement().$('button#inside-shadow');
3 await button.click();
```

## Custom Web Element with Test Attribute – Interview Answer

“A Custom Web Element with a Test Attribute is an HTML element that includes a unique attribute like `data-test` or `data-testid`, used specifically for automation testing. It helps tools like Playwright or Selenium reliably locate elements without depending on CSS classes or text, which may change.”

```
await page.locator('[data-test="login-button"]').click();
```

#### 💡 Why It's Useful:

- Makes tests more stable and less brittle
- Avoids breaking tests due to UI changes
- Improves readability and maintainability of test scripts

## 22) Overview of Element State

Element State refers to the current condition or behavior of a web element, such as whether it's visible, enabled, checked, editable, or hidden. In testing, checking element states helps ensure the UI behaves as expected before performing actions.”

#### 🔧 Common Element States in Playwright:

- visible – Element is displayed on the page
- enabled – Element is interactive (not disabled)
- checked – Checkbox or radio is selected
- editable – Input field can be typed into
- hidden – Element is not visible

```
await expect(page.locator('#submit')).toBeVisible();
await expect(page.locator('#submit')).toBeEnabled();
await expect(page.locator('#terms')).toBeChecked();
```

## 23) Performing Element and Mouse actions (Click Action, Right Click etc)

### Element and Mouse Actions

“Element and mouse actions in Playwright are used to simulate user interactions like clicking, right-clicking, hovering, and dragging. These actions help validate UI behavior and user workflows during automated testing.”

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| Click Action  | <code>await page.locator('#submit').click();</code>                         |
| Right Click   | <code>await page.locator('#menu').click({ button: 'right' });</code>        |
| Hover         | <code>await page.locator('#profile icon').hover();</code>                   |
| Double click  | <code>await page.locator('#edit').dblclick();</code>                        |
| Drag and Drop | <code>await page.locator('#source').dragTo(page.locator('#target'));</code> |

#### Why It's Important:

- Simulates real user behavior
- Helps test dynamic UI elements
- Ensures workflows like form submission, navigation, and context menus work correctly.

#### 24) How to type text into an input field ?

“To type text into an input field in Playwright, we use the fill() or type() method. fill() replaces any existing text, while type() types character by character, simulating real user input.”

|                                                          |                                                             |
|----------------------------------------------------------|-------------------------------------------------------------|
| Using fill()for faster input                             | <b>await page.locator('#username').fill('Reethika123');</b> |
| Using type() when testing typing delays or input events. | <b>await page.locator('#username').type('Reethika123');</b> |

#### 25) How to Handle static Dropdowns and Suggestion Drop down

Handling Static and Suggestion Dropdowns – Interview Answer

“In Playwright, static dropdowns are handled by selecting predefined options using the selectOption() method. Suggestion dropdowns, which are dynamic and appear based on user input, are handled by typing into the input field and selecting the suggestion using locators.”

|                    |                                                                                                                                                      |  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Static dropdown    | <b>await page.locator('#country').selectOption('India');</b>                                                                                         |  |
| Suggested dropdown | <b>await page.locator('#search').type('Playwright');</b><br><b>await page.locator('.suggestion-item &gt;&gt; text=Playwright Tutorial').click();</b> |  |

Tip:

- Use selectOption() for <select> tags.
- Use type() + click() for dynamic dropdowns or autocomplete fields.

#### 26) Handling Web Table with Playwright

“In Playwright, we handle web tables by locating rows and columns using CSS or XPath selectors. We can loop through rows, extract cell data, and perform actions like clicking or validating values.”



## Use Cases:

- Validating table content
- Clicking buttons inside rows
- Filtering or sorting data

```
const rows = await page.locator('table#employeeTable tbody tr');
const count = await rows.count();

for (let i = 0; i < count; i++) {
  const name = await rows.nth(i).locator('td').nth(0).textContent();
  const role = await rows.nth(i).locator('td').nth(1).textContent();
  console.log(`Name: ${name}, Role: ${role}`);
}
```

## 27) Handling Checkbox and Radio Button

In Playwright, checkboxes and radio buttons are handled using methods like `check()`, `unchecked()`, and `click()`. These methods help simulate user selection and validate form behavior.”

```
1 await page.locator('#subscribe').check();    // To check
2 await page.locator('#subscribe').unchecked(); // To uncheck
3
```

Radio Button



```
1 await page.locator('input[type="radio"][value="female"]').check();
```

## 28) Handling Alerts, Confirms and Prompts / Dialogs

“In Playwright, browser dialogs like alerts, confirms, and prompts are handled using the `page.on('dialog')` event. We can listen to the dialog, read its message, and choose to accept or dismiss it.

## Handling an Alert

```
1 page.on('dialog', async (dialog) => {
2   console.log(dialog.message());
3   await dialog.accept(); // or dialog.dismiss()
4 });
5
6 await page.click('#trigger-alert');
```

## Handling a Prompt

```
1 page.on('dialog', async (dialog) => {
2   await dialog.accept('Reethika'); // Send input to prompt
3 });
4
5 await page.click('#trigger-prompt');
```

## 29) Fetching Text from the field Using Playwright

“In Playwright, we use the `textContent()` or `inputValue()` method to fetch text from elements. `textContent()` is used for static text like labels or headings, while `inputValue()` is used for input fields.”

### Tip:

- Use `textContent()` for elements like `<div>`, `<span>`, `<p>`, etc.
- Use `inputValue()` for `<input>` and `<textarea>` fields.

### ✓ Fetching Static Text

```
1 const label = await page.locator('#welcome-msg').textContent();
2 console.log(label);
```

### ✓ Fetching Text from Input Field

```
const username = await page.locator('#username').inputValue();
console.log(username);
```

## 30) Strategy on handling Calenders , Forms, Dropdowns and Clock

## Calendars

“For calendar widgets, we interact by clicking date pickers, navigating months, and selecting dates using locators. Sometimes, we input dates directly if the field allows typing.

---

```
1  await page.locator('#datePicker').click();
2  await page.locator('.day >> text=18').click(); // Selects 18th
3
```

## Forms

“Forms are handled by filling input fields, selecting options, checking boxes, and submitting using fill(), check(), and click() methods.”

```
await page.locator('#name').fill('Reethika');
await page.locator('#email').fill('reethika@example.com');
await page.locator('#submit').click();
```

## Dropdowns

“Static dropdowns use selectOption(), while dynamic suggestion dropdowns are handled by typing and selecting from the list.”

```
await page.locator('#country').selectOption('India');
```

## Clocks / Time Pickers

“Time pickers are handled by selecting hours and minutes from dropdowns or clicking time slots. If allowed, we can also type the time directly.”

```
await page.locator('#timePicker').click();
await page.locator('.time-slot >> text=10:30 AM').click();
```

## 31) How to fill out a form using Playwright

“In Playwright, we fill out a form by using methods like fill() for text inputs, check() for checkboxes, selectOption() for dropdowns, and click() to submit. These simulate user interactions and help validate form functionality.”

```
1 await page.locator('#name').fill('Reethika');
2 await page.locator('#email').fill('reethika@example.com');
3 await page.locator('#subscribe').check();
4 await page.locator('#country').selectOption('India');
5 await page.locator('#submit').click();
6
```

---

### 32) Nested Frames

“Nested frames are frames (or iframes) embedded within other frames on a webpage. In Playwright, we handle them by first accessing the parent frame, then navigating to the child frame using `frameLocator()`.”

```
const parentFrame = page.frameLocator('#parent-frame');
await parentFrame.frameLocator('#child-frame').locator('#submit').click();
```

### 33) Frame Selectors

“Frame selectors in Playwright are used to interact with elements inside `<iframe>` or `<frame>` tags. We use `frameLocator()` to target the frame, and then chain locators to access elements within it.

```
const frame = page.frameLocator('#my-frame');

await frame.locator('#submit-button').click;
```

### 34) Child Frame Navigation

“In Playwright, child frame navigation means accessing and interacting with elements inside an iframe that is nested within the main page. We use `frameLocator()` to target the child frame and then chain locators to perform actions inside it.”

```
const childFrame = page.frameLocator('#child-frame');

await childFrame.locator('#submit-button').click();
```

For **nested frames**, chain `frameLocator()`:

```
page.frameLocator('#parent-frame').frameLocator('#child-frame').locator('#element');
```

35) **What are Hooks in Playwright and its uses?** → “Hooks in Playwright are special functions used to run code before or after tests. They help set up preconditions like launching the browser, logging in, or cleaning up after tests. Common hooks include `beforeAll`, `beforeEach`, `afterEach`, and `afterAll`.”

### Hook Types:

- beforeAll – Runs once before all tests
- beforeEach – Runs before each test
- afterEach – Runs after each test
- afterAll – Runs once after all tests

| Hook Type  | Runs When             | Use Case                                                             |
|------------|-----------------------|----------------------------------------------------------------------|
| beforeAll  | Once before all tests | Set up shared resources (e.g., browser launch, login, DB connection) |
| beforeEach | Before every test     | Prepare test environment (e.g., navigate to page, reset data)        |
| afterEach  | After every test      | Clean up (e.g., clear cookies, log results, reset mocks)             |
| afterAll   | Once after all tests  | Tear down shared resources (e.g., close browser, disconnect DB)      |

```
test.beforeAll(async () => {
  console.log('Setup before all tests');
});

test.beforeEach(async ({ page }) => {
  await page.goto('https://example.com');
});

test.afterEach(async () => {
  console.log('Cleanup after each test');
});

test.afterAll(async () => {
  console.log('Teardown after all tests');
});
```

- ☐ Use beforeAll and afterAll for **global setup/teardown**.
- ☐ Use beforeEach and afterEach for **test-level isolation**.

### 36) Handling Mouse Actions with Playwright Hooks

“In Playwright, we can use hooks like beforeEach to set up mouse actions such as hover, click, or drag before each test. This ensures consistent interaction with UI elements and helps simulate real user behavior.”

```

1 import { test } from '@playwright/test';
2
3 test.beforeEach(async ({ page }) => {
4   await page.goto('https://example.com');
5   await page.locator('#menu').hover(); // Hover before each test
6 });
7
8 test('Click action', async ({ page }) => {
9   await page.locator('#start-button').click();
10 });
11
12 test('Drag and drop', async ({ page }) => {
13   await page.locator('#item').dragTo(page.locator('#target'));
14 });
15

```

### 37) What are assertions in Playwright?

“Assertions in Playwright are used to verify that the application behaves as expected. They help validate conditions like element visibility, text content, URL, or input values during tests.”

| Assertion                | Example                                                                            |
|--------------------------|------------------------------------------------------------------------------------|
| Element is visible       | <code>await expect(page.locator('#login')).toBeVisible();</code>                   |
| Element contains text    | <code>await expect(page.locator('h1')).toHaveText('Welcome');</code>               |
| URL is correct           | <code>await expect(page).toHaveURL(/dashboard/);</code>                            |
| Input has specific value | <code>await expect(page.locator('#email')).toHaveValue('test@example.com');</code> |
| Checkbox is checked      | <code>await expect(page.locator('#agree')).toBeChecked();</code>                   |

### 38) Steps to Use Built-in Assertions and Exception Timeout?

Steps to Use Built-in Assertions in Playwright

Playwright provides a built-in expect API for assertions. Here's how to use it:

| Step | Action                             | Example                                                       |
|------|------------------------------------|---------------------------------------------------------------|
| 1    | Import expect from Playwright Test | <code>import { test, expect } from '@playwright/test';</code> |
| 2    | Locate the element                 | <code>const button = page.locator('#submit');</code>          |
| 3    | Apply assertion                    | <code>await expect(button).toBeVisible();</code>              |
| 4    | Use other assertions as needed     | <code>await expect(page).toHaveURL(/dashboard/);</code>       |

```

await expect(locator).toBeVisible();
await expect(locator).toHaveText('Welcome');
await expect(locator).toHaveValue('Reethika');
await expect(page).toHaveURL(/home/);

```

## ⌚ Exception Timeout in Assertions

“Timeout is the maximum time Playwright waits for an assertion to pass before throwing an error.”

### ✅ Set Timeout for Assertions:

```
1 await expect(page.locator('#submit')).toBeVisible({ timeout: 5000 });
2
```

## 39) What is custom validation logic and its need?

### Custom Validation Logic

“Custom validation logic in Playwright refers to writing our own conditions to verify specific behaviors or data that built-in assertions may not cover. It’s useful when we need to validate complex UI states, dynamic content, or business-specific rules.”

### Why It’s Needed:

- Built-in assertions may not cover **custom business rules**
- Useful for **dynamic validations** (e.g., price ranges, date formats)
- Helps in **complex workflows** like multi-step forms or conditional.

#### 📄 Steps to Implement Custom Validation Logic in Playwright

| Step | Action                              | Purpose                                                                                    |
|------|-------------------------------------|--------------------------------------------------------------------------------------------|
| 1    | Locate the element                  | Use <code>locator()</code> to target the element you want to validate                      |
| 2    | Extract the value or text           | Use methods like <code>textContent()</code> or <code>inputValue()</code>                   |
| 3    | Apply custom logic                  | Use JavaScript conditions to validate against business rules                               |
| 4    | Throw error if validation fails     | Use <code>throw new Error()</code> to fail the test manually                               |
| 5    | Optionally log or assert the result | Use <code>console.log()</code> or <code>expect()</code> for debugging or additional checks |

## 40) How to integrate custom validation logic into test scripts?

Custom Web Element (Test Attribute)

| Step | Action                              | Purpose                                                                                    |
|------|-------------------------------------|--------------------------------------------------------------------------------------------|
| 1    | Locate the element                  | Use <code>locator()</code> to target the element you want to validate                      |
| 2    | Extract the value or text           | Use <code>textContent()</code> or <code>inputValue()</code> to get the actual data         |
| 3    | Apply custom logic                  | Use JavaScript conditions to validate against business rules                               |
| 4    | Throw error if validation fails     | Use <code>throw new Error()</code> to fail the test manually                               |
| 5    | Optionally log or assert the result | Use <code>console.log()</code> or <code>expect()</code> for debugging or additional checks |

## Use Cases:

- Validating dynamic values (e.g., prices, dates, totals)
- Checking custom formats (e.g., phone numbers, emails)
- Enforcing business rules (e.g., discount limits, form logic)

## Example: Custom Validation in a Test Script

Custom Web Element (Test Attribute)

```
1 import { test, expect } from '@playwright/test';
2
3 test('Validate price is within limit', async ({ page }) => {
4   await page.goto('https://example.com');
5
6   const priceText = await page.locator('#price').textContent();
7   const price = parseFloat(priceText?.replace('$', ''));
8
9   if (price > 1000) {
10    throw new Error(`Price too high: ${price}`);
11  }
12
13  console.log(`Validated price: ${price}`);
14 };
```

## Run All Tests

### How to Run All Tests in Playwright

“To run all tests in Playwright, we use the Playwright Test Runner with the command `npx playwright test`. This automatically discovers and runs all test files in the project.”

Custom Web Element (Test Attribute)

#### Steps to Run All Tests:

| Step | Action                                                                           |
|------|----------------------------------------------------------------------------------|
| 1    | Open terminal in your project directory                                          |
| 2    | Run the command: <code>npx playwright test</code>                                |
| 3    | Playwright will execute all <code>.spec.ts</code> or <code>.test.ts</code> files |
| 4    | View results in the terminal or HTML report                                      |

Generate HTML Report-> **`npx playwright test --reporter=html`**

Open the report -> **`npx playwright show-report`**



## 41) Execute a Single Test File

### How to Run a Single Test File in Playwright

“To run a specific test file in Playwright, use the command `npx playwright test <file-path>`. This helps isolate and debug individual test cases.”

Command -> **`npx playwright test tests/login.spec.ts`**

Use `--headed` to run in headed mode: -> **`npx playwright test tests/login.spec.ts --headed`**

Use `--debug` to pause and inspect: -> **`npx playwright test tests/login.spec.ts --debug`**

## 42) Execute tests in parallel , And Serial

### Parallel:

“Playwright runs tests in parallel by default to speed up execution. To run tests serially, we use the `test.describe.configure({ mode: 'serial' })` setting inside a test suite.”

|                                         |
|-----------------------------------------|
| <b><code>npx playwright test</code></b> |
|-----------------------------------------|

### Serial:

Custom Web Element (Test Attribute)

```
1 import { test } from '@playwright/test';
2
3 test.describe.configure({ mode: 'serial' });
4
5 test.describe('Serial Tests', () => {
6   test('Test A', async ({ page }) => {
7     // Runs first
8   });
9
10  test('Test B', async ({ page }) => {
11    // Runs after Test A
12  });
13 });
```

### Tip:

- Use parallel for speed.
- Use serial when tests depend on each other or share state.

Execute a Set of Test Files

### How to Run a Set of Test Files in Playwright

“To run a specific set of test files, we pass their paths to the Playwright test command. This helps in running only selected tests during development or debugging.”

#### 43) Execute Tests in a Particular Browser with out the Configuration

Run Tests in a Specific Browser (e.g., Chromium, Firefox, WebKit) - >

```
npx playwright test --project=chromium
```

Run a Specific Test File in a Specific Browser

```
npx playwright test tests/example.spec.ts --project=firefox
```

Run Tests Headless or Headed

```
npx playwright test --project=chromium --headed
```

#### Execute tests in All browser with out the Configuration

Option 1: Run Tests in All Browsers (Default Projects) ->

```
npx playwright test --project=chromium --project=firefox --project=webkit
```

Option 2: Run All Tests Without Specifying Projects (if tests are browser-specific)

```
npx playwright test
```

Optional: Run Headed or Headless

```
npx playwright test --project=chromium --project=firefox --project=webkit --headed
```

```
npx playwright test --project=chromium --project=firefox --project=webkit --headless
```

#### 44) Emulation Options( Timezone, Geo location, Viewport ...)

Playwright allows you to emulate various browser environments **without needing a configuration file** by using the `test.use()` method directly in your test files or by passing options programmatically.

| Feature     | How to Emulate                                         |
|-------------|--------------------------------------------------------|
| Timezone    | timezoneId: 'Asia/Kolkata'                             |
| Geolocation | geolocation: { latitude: 13.0827, longitude: 80.2707 } |
| Viewport    | viewport: { width: 1280, height: 720 }                 |
| Locale      | locale: 'en-IN'                                        |
| Permissions | permissions: ['geolocation']                           |
| User Agent  | userAgent: 'custom-agent-string'                       |

### ✓ Example: Emulate All in a Test File

```
1 import { test, expect } from '@playwright/test';
2
3 test.use({
4   timezoneId: 'Asia/Kolkata',
5   geolocation: { latitude: 13.0827, longitude: 80.2707 },
6   permissions: ['geolocation'],
7   viewport: { width: 1280, height: 720 },
8   locale: 'en-IN',
9   userAgent: 'custom-agent-string',
10 });
```

Execute a Set of Test Files

```
12 test('Emulated environment test', async ({ page }) => {
13   await page.goto('https://www.google.com');
14   await expect(page).toHaveTitle(/Google/);
15 });
16
```

#### 45) What are Test Retries in Playwright?

Retries automatically re-run a test if it fails. This is useful for flaky tests that might fail due to temporary issues like slow network or animations.

```
Ts ^ Copy

import { defineConfig } from '@playwright/test';

export default defineConfig({
  retries: 2, // Retries failed tests up to 2 times
});
```

You can also run with retries from the command line:

```
Bash ^ Copy

npx playwright test --retries=2
```

#### 46) What are Timeouts in Playwright?

**Timeouts** define how long Playwright should wait before failing a test or action. This helps avoid tests hanging forever.

##### ◆ Example of setting a timeout for a test:

```
Ts ^ Copy

test('loads dashboard', async ({ page }) => {
  await page.goto('https://example.com', { timeout: 10000 }); // waits max 1
});
```

##### ◆ Set global timeout in config:

```
Ts ^ Copy

export default defineConfig({
  timeout: 30000, // 30 seconds for each test
});
```

#### 47) What is Page Fixtures ? How to use it in Execution ?

“Page fixture in Playwright gives each test a clean browser tab. It simplifies setup and ensures tests don’t interfere with each other. I just use { page } in my test function, and Playwright handles the rest.”

A **fixture** is a reusable piece of setup code that provides context or resources to your tests. Playwright comes with several built-in fixtures like:

- page: Represents a single tab in a browser.
- browser: The browser instance.
- context: A browser context (like an incognito window).

```
Ts ^  
  
import { test, expect } from '@playwright/test';  
  
test('Check page title', async ({ page }) => {  
  await page.goto('https://playwright.dev/');  
  await expect(page).toHaveTitle(/Playwright/);  
});
```

#### 48) Emulating Mobile Devices

##### What is Mobile Device Emulation?

Mobile device emulation is the process of simulating the behavior and characteristics of a mobile device (like screen size, resolution, touch input, and user agent) on a desktop browser or automation tool. It helps test how a website or web app behaves on different mobile devices without needing physical devices.

##### Why is it Useful?

- Test responsive design and layout
- Simulate touch interactions
- Validate mobile-specific features
- Save cost and time (no need for real devices)
- Easily test across multiple device types

```

1  const { chromium, devices } = require('playwright');
2  const iPhone = devices['iPhone 12'];
3
4  const browser = await chromium.launch();
5  const context = await browser.newContext({
6    ...iPhone
7  });
8  const page = await context.newPage();
9  await page.goto('https://example.com');

```

#### 49) What is Component Testing and How to use it in Playwright ? (Beta)

Component Testing is a technique used to test individual UI components in isolation, ensuring they behave correctly without relying on the entire application. In Playwright (Beta), this feature allows developers to mount components directly in a real browser, interact with them using Playwright's powerful APIs, and validate their behavior, appearance, and events.

Unlike traditional unit tests that run in simulated environments, Playwright's component tests run in actual browser contexts, offering more accurate and reliable results.

##### How to Use Component Testing in Playwright (Beta)

Here's a step-by-step breakdown you can confidently share in an interview:

##### 1. Setup Playwright for Component Testing

Use the CLI to initialize a project with component testing support:

```
npm init playwright@latest -- --ct
```

**This sets up:**

- **playwright/index.html** → where components are rendered
- **playwright/index.ts** → for runtime setup (themes, global styles)

##### 2. Write a Component Test

```
import { test, expect } from '@playwright/experimental-ct-react';
import Button from './Button';

test('should trigger click event', async ({ mount }) => {
  let clicked = false;

  const component = await mount(
    <Button title="Submit" onClick={() => (clicked = true)} />
  );

  await expect(component).toContainText('Submit');
  await component.click();
  expect(clicked).toBeTruthy();
});
```

### 3. Run the Tests

You can run tests via command line or VS Code extension

`npm run test-ct`

### 4. Leverage Playwright Features

Component testing supports:

-  Real browser rendering
-  Event simulation
-  Visual regression testing
-  Network mocking
-  Parallel execution

### 50) Screenshot capture techniques

#### Full Page Screenshot

Ts ^

 Copy

```
await page.screenshot({ path: 'fullpage.png', fullPage: true });
```

#### Element Screenshot

Ts ^

 Copy

```
await page.locator('.header').screenshot({ path: 'element.png' });
```

"Playwright provides powerful screenshot capabilities for UI validation, debugging, and visual regression testing. Using the `page.screenshot()` method, we can capture full-page, viewport, or element-specific screenshots across Chromium, Firefox, and WebKit browsers.

## 51) Screenshots embedding in the report

"In Playwright, we can embed screenshots directly into the test report using the `testInfo.attach()` method. This is especially useful for debugging failed tests and enhancing report visibility."

```
test('Login page test', async ({ page }, testInfo) => {  
  await page.goto('https://example.com/login');  
  const screenshot = await page.screenshot();  
  testInfo.attach('Login Page', {  
    body: screenshot,  
    contentType: 'image/png',  
  });  
});
```

- `testInfo` is available as the second argument in every test
- `attach()` embeds the screenshot with a label and MIME type
- Screenshots appear in the Playwright HTML report when you run:

Bash ^

`npx playwright test --reporter html`

## 52) Comparing visual snapshots in playwright?

"In Playwright, we use `expect(page).toHaveScreenshot()` to capture and compare visual snapshots of a page or element. It helps detect UI regressions by comparing the current screenshot with a baseline image. If there's a mismatch, Playwright highlights the differences. This is useful for ensuring consistent UI across releases."

```
test('Login page visual test', async ({ page }) => {  
  
  await page.goto('https://example.com/login');  
  
  await expect(page).toHaveScreenshot('login-page.png');  
  
});
```

## 53) Comprehensive Test Coverage for end-to-end tests

"Comprehensive test coverage in E2E testing means validating all critical user flows, edge cases, and integrations across the application. In Playwright, we achieve this by designing modular tests that cover login, navigation, form submissions, error handling, and role-based access. It ensures confidence in the app's behavior from the user's perspective."

```
test('User can log in and view dashboard', async ({ page }) => {
```



```
await page.goto('https://example.com/login');

await page.fill('#username', 'testuser');

await page.fill('#password', 'password123');

await page.click('button[type="submit"]');

await expect(page).toHaveURL('https://example.com/dashboard');

await expect(page.locator('h1')).toHaveText('Welcome, testuser');

});
```

### 53) tsconfig.json for Playwright

In a Playwright project using TypeScript, tsconfig.json configures how TypeScript compiles the test files. It defines settings like the target JavaScript version, module system, and type definitions. This ensures type safety, better IntelliSense, and smoother integration with Playwright APIs.”

```
{

  "compilerOptions": {

    "target": "ESNext",

    "module": "CommonJS",

    "lib": ["ESNext", "DOM"],

    "moduleResolution": "Node",

    "strict": true,

    "esModuleInterop": true,

    "skipLibCheck": true,

    "forceConsistentCasingInFileNames": true,

    "types": ["playwright"]

  },

  "include": ["tests/**/*.ts"]

}
```

“For example, we set types: ["playwright"] to include Playwright’s type definitions, and include: ["tests/\*\*/\*.ts"] to compile only our test files. This helps maintain clean and efficient test code.”

#### 54) Handling complex user interactions

“Playwright handles complex user interactions like drag-and-drop, hover, keyboard shortcuts, and multi-step forms using its powerful API. We can simulate real user behavior with high accuracy, including chained actions and conditional flows.”

```
await page.hover('#menu');

await page.click('#submenu-item');

await page.dragAndDrop('#source', '#target');

await page.keyboard.press('Control+A');
```

#### 55) Event Handling with Playwright

“Playwright allows us to handle browser events like dialogs, downloads, popups, and network requests using event listeners. This helps us test how the app responds to dynamic actions like alerts, file downloads, or new tabs.”

```
1  page.on('dialog', async dialog => {
2    console.log(`Dialog message: ${dialog.message()}`);
3    await dialog.accept();
4  });
5
6  page.on('download', async download => {
7    const path = await download.path();
8    console.log(`File downloaded to: ${path}`);
9  });
```

#### 56) Test Parameterization with different data sets

“In Playwright, we use test parameterization to run the same test with multiple data sets. This helps validate different input combinations efficiently. We achieve this using test.describe() with forEach() or by using test.each() from the Playwright Test runner.”

```
3 const loginData = [
4   { username: 'user1', password: 'pass1' },
5   { username: 'user2', password: 'pass2' },
6 ];
7
8 loginData.forEach(({ username, password }) => {
9   test(`Login test for ${username}`, async ({ page }) => {
10     await page.goto('https://example.com/login');
11     await page.fill('#username', username);
12     await page.fill('#password', password);
13     await page.click('button[type="submit"]');
14     await expect(page).toHaveURL(/dashboard/);
15   });
16 });
```

## 57) Data-Driven Frameworks

“A Data-Driven Framework separates test logic from test data, allowing us to run the same test with multiple input sets. This improves reusability, scalability, and coverage. In Playwright, we can achieve this by reading data from external sources like JSON, CSV, or Excel, and looping through them to feed into our tests.”

```
const logindata1=[
  {username:"standard_user", password:"secret_sauce"},
  {username:"locked_out_user", password:"secret_sauce"},
  {username:"problem_user", password:"secret_sauce"},
  {username:"performance_glitch_user", password:"secret_sauce"}
]

const obj ={name:"Sairaj",age:'27'}

console.log(obj.name)//Sairaj
for(const data of logindata1){
  //standard_user
  test(`login test for ${data.username}`, async()=>{
    const browser=await chromium.launch({headless:false});
    const page=await browser.newPage();
    await page.goto('https://www.saucedemo.com/v1/');
    await page.fill('#user-name',data.username);
    await page.fill('#password',data.password);
    await page.waitForTimeout(3000);
    await page.click('#login-button');
  })
}
```

### 58) Data Loading from external sources (e.g. Excel, CSV, JSON):

“In Playwright, we can load test data from external sources like JSON, CSV, or Excel to support data-driven testing. This allows us to run the same test with multiple input sets, improving coverage and flexibility. We typically use Node.js modules like fs for JSON/CSV or xlsx for Excel.”

```
1 import testData from './data/loginData.json';
2
3 testData.forEach(({ username, password }) => {
4   test(`Login test for ${username}`, async ({ page }) => {
5     await page.goto('https://example.com/login');
6     await page.fill('#username', username);
7     await page.fill('#password', password);
8     await page.click('button[type="submit"]');
9     await expect(page).toHaveURL(/dashboard/);
10   });
11 });
```

```
1 import * as XLSX from 'xlsx';
2
3 const workbook = XLSX.readFile('./data/loginData.xlsx');
4 const sheet = workbook.Sheets[workbook.SheetNames[0]];
5 const testData = XLSX.utils.sheet_to_json(sheet);
6
7 testData.forEach(({ Username, Password }) => {
8   test(`Login test for ${Username}`, async ({ page }) => {
9     await page.goto('https://example.com/login');
10    await page.fill('#username', Username);
11    await page.fill('#password', Password);
12    await page.click('button[type="submit"]');
13    await expect(page).toHaveURL(/dashboard/);
14  });
15 });
```

## 59) Parameterizing the Test Data from JSON

“In Playwright, we can parameterize test data by loading it from a JSON file. This allows us to run the same test with multiple data sets, improving coverage and reducing duplication. We use require() or import to load the JSON and loop through the data using forEach().”

```

1 import { test, expect } from '@playwright/test';
2 import loginData from './data/loginData.json';
3
4 loginData.forEach(({ username, password }) => {
5   test(`Login test for ${username}`, async ({ page }) => {
6     await page.goto('https://example.com/login');
7     await page.fill('#username', username);
8     await page.fill('#password', password);
9     await page.click('button[type="submit"]');
10    await expect(page).toHaveURL(/dashboard/);
11  });
12 });

```

## 60) Data managing in the end to end tests

“In E2E testing, data management involves organizing test data for setup, execution, and validation. We use external files like JSON, CSV, or Excel to store reusable data sets. This supports data-driven testing, improves maintainability, and separates test logic from data. We also use fixtures or environment-specific data to simulate real-world scenarios.”

```

1 import testData from './data/userData.json';
2
3 testData.forEach(({ username, password }) => {
4   test(`Login test for ${username}`, async ({ page }) => {
5     await page.goto('https://example.com/login');
6     await page.fill('#username', username);
7     await page.fill('#password', password);
8     await page.click('button[type="submit"]');
9     await expect(page).toHaveURL(/dashboard/);
10   });
11 });

```

## 70) Screenshot and Record Execution

Use:

{ video:on,

trace:on,

screenshot:on

}

## 71) Generate log mechanism

To demonstrate a **logging mechanism in Playwright** during an interview, you can showcase how to capture and log various events such as console messages, network requests, and test execution details. Here's a clean and interview-ready example using **TypeScript** with Playwright Test:

### ✓ Logging Mechanism in Playwright (TypeScript)

#### 1. Log Console Messages from the Browser

```
1 import { test, expect } from '@playwright/test';
2
3 test('Log console messages', async ({ page }) => {
4   page.on('console', msg => {
5     console.log(`[Browser Console] ${msg.type().toUpperCase():} ${msg.text()}`);
6   });
7
8   await page.goto('https://example.com');
9 }
```

#### 2. Log Network Requests and Responses

```
1 test('Log network activity', async ({ page }) => {
2   page.on('request', request => {
3     console.log(`[Request] ${request.method()} ${request.url()}`);
4   });
5
6   page.on('response', response => {
7     console.log(`[Response] ${response.status()} ${response.url()}`);
8   });
9
10  await page.goto('https://example.com');
11 }
```



### 3. Custom Logger for Test Steps

You can create a simple logger utility:

```
1 function logStep(step: string) {  
2   console.log(`[Test Step] ${step}`);  
3 }
```

Use it as

```
1 test('Login flow with logging', async ({ page }) => {  
2   logStep('Navigating to login page');  
3   await page.goto('https://example.com/login');  
4  
5   logStep('Filling in credentials');  
6   await page.fill('#username', 'testuser');  
7   await page.fill('#password', 'password123');  
8  
9   logStep('Clicking login button');  
10  await page.click('#login');  
11  
12  logStep('Verifying login success');  
13  await expect(page.locator('#welcome')).toBeVisible();  
14 });
```

### 4. Write Logs to a File (Node.js fs module)



```
1 import fs from 'fs';  
2  
3 function writeLog(message: string) {  
4   fs.appendFileSync('test-log.txt', `${new Date().toISOString()} - ${message}\n`);  
5 }
```

Use it like

```
1 page.on('console', msg => {  
2   writeLog(`[Console] ${msg.text()}`);  
3 });
```

72)What is Page Object Model?

A design pattern in test automation. It creates a separate class for each web page of the application. Each page contains the locators and methods related to that page.

Why is pom useful?

Reusability, Maintainability, Readability, Less Duplication, Scalability, Better Structure.

73) What is custom validation?

In **Playwright**, **custom validation** refers to writing your own logic to verify that a certain condition or behavior is met during a test. This is useful when built-in assertions like `expect(locator).toHaveText()` or `expect(locator).toBeVisible()` aren't enough for your specific scenario.

```
import { test } from '@playwright/test';

test('custom validation - heading text', async ({ page }) => {
  await page.goto('https://example.com');

  const heading = await page.locator('h1').textContent();

  if (heading !== 'Welcome') {
    throw new Error('Heading is not correct!');
  }
});
```

**Fixtures** in Playwright are reusable setup and teardown code that help manage test environments. They allow you to define things like:

- Browser and page setup
- Logged-in sessions
- Test data
- Custom utilities

Playwright uses fixtures to make tests **clean**, **modular**, and **efficient**.

---

### Advantages of Fixtures (Easy & Interview-Friendly)

| Advantage       | Explanation                                     |
|-----------------|-------------------------------------------------|
| ✅ Reusability   | Write setup code once, use it in multiple tests |
| ✅ Cleaner tests | Keeps test logic focused on what you're testing |



| Advantage            | Explanation                                                              |
|----------------------|--------------------------------------------------------------------------|
| ✓ Custom setup       | You can create your own fixtures for login, API mocks, etc.              |
| ✓ Automatic teardown | Playwright handles cleanup after each test automatically                 |
| ✓ Parallel testing   | Fixtures help isolate tests, making parallel execution safe and reliable |

Import {test as baseTest} from '@playwright/test'

Ex: type pages

```
{
```

```
  landingpage:Landingpage
```

```
  Homepage:homepage
```

```
}
```

```
Const testpages=baseTest.extend<page>({
```

```
{
```

```
  landingpage:async({page},use)=>
```

```
{
```

```
    await use(new LandingPage(page));
```

```
  })
```

```
export const test=testpages;
```

```
export const expect=testpages.expect;
```

in the original playwright we need to write->

```
import {test,expect} from '../fixtures/pomFixtures';
```

```

1 // my playwright.config.ts
2 import { test as base } from '@playwright/test';
3
4 const test = base.extend({
5   loggedInPage: async ({ page }, use) => {
6     await page.goto('https://example.com/login');
7     await page.fill('#username', 'user');
8     await page.fill('#password', 'pass');
9     await page.click('button#login');
10    await use(page); // Pass the logged-in page to the test
11  },
12 });
13
14 export { test };

```

Reporters in playwright -> html, lists, dot, line, json

### When do we use Allure Report vs HTML Report in Playwright?

#### ✅ Use Allure Report when:

- You want detailed, interactive reports with:
  - Step-by-step logs
  - Screenshots
  - Attachments
  - Test history
- You need to share reports with teams or integrate with CI/CD tools like Jenkins.
- You want to track flaky tests, retries, and trends over time.

 *"I use Allure when I need rich, professional reports for team visibility or CI pipelines."*

---

#### ✅ Use HTML Report when:

- You want a quick, built-in report with no extra setup.
- You're debugging locally or running tests manually.
- You don't need advanced features like attachments or history.

 *"I use the default HTML report for fast local debugging or small projects."*

---

### Bonus Tip for Interview:

**“In my projects, I use HTML reports during development and switch to Allure for CI/CD and stakeholder reporting.”**

## Allure Report

- Allure Report is an open-source reporting framework primarily used in the context of test automation for various programming languages, including Java, JavaScript, Python, and others.
- It provides visually appealing and informative reports that help testers, developers, and stakeholders better understand the results of test executions.
- Allure Report was originally developed for use with the TestNG testing framework but has since been extended to support other testing frameworks as well.

## Allure VS Inbuilt HTML Report

- **Customisation:** Allure provides more extensive customization options for report generation. You can tailor the report's appearance and content to match your project's specific requirements.
- **Rich Visuals:** Allure offers visually appealing reports with various charts and graphs that can help you quickly identify trends and issues in your test results.
- **History & Trends:** Allure allows you to view historical test results and trends, making it easier to track improvements or regressions in your tests over time.
- **Integration:** Allure supports integration with other testing frameworks and tools, making it suitable for projects with diverse toolchains.
- **Attachments & Plugins:** Allure supports attaching additional files or artifacts to your test reports. Also Allure has a plugin system that allows you to extend its functionality and integrate it with other tools or services as needed.



API (Full form **Application Programming Interface**) enables communication and data exchange between two separate software systems



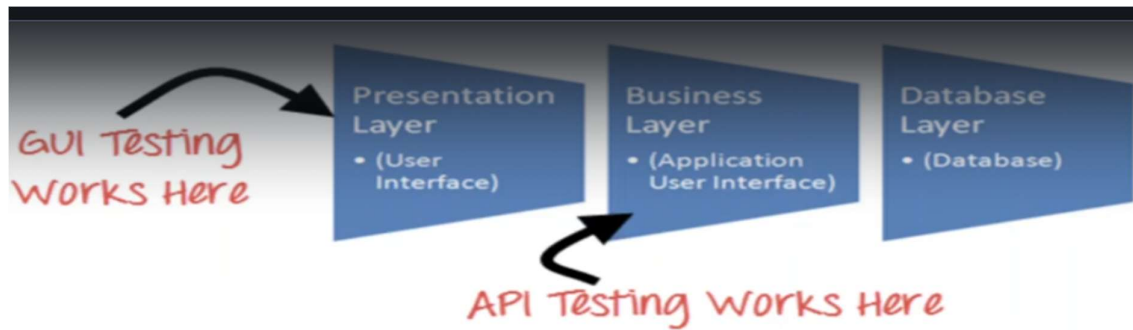
A software system implementing an API contains functions/sub-routines which can be executed by another software system



In simple english, API is the backbone of Web UI. Whenever we make any action in UI, an API call will be send to server



Without API we cannot perform any action in the UI



- ➔ API TESTING is a software testing type that validates Application Programming Interfaces (APIs)
- ➔ The purpose of API Testing is to check the functionality, reliability, performance, and security of the programming interfaces

- ➔ In API Testing, instead of using standard user inputs(keyboard) and outputs, you use software to send calls to the API, get output, and note down the system's response

- ➔ API tests are very different from GUI Tests and won't concentrate on the look and feel of an application. It mainly concentrates on the business logic layer of the software architecture.

- ➔ In order to test an API, you will need
  1. Use Testing Tool to drive the API
  2. Write your own code to test the API

## TOOLS TO PERFORM API TESTING



## GET Method

- **Purpose:** Retrieve data from the server.
- **Example Use Case:** Viewing a webpage, fetching user details, searching something.
- **Characteristics:**
  - Data is sent in the **URL** (as query parameters).
  - Can be **bookmarked** and cached.
  - **Not secure** for sensitive data (e.g., passwords).
  - Typically used for **read-only** operations.

```
apiTest.spects > test('Get user Details using GET API') callback
import { test, expect } from '@playwright/test';

test('Get user Details using GET API', async ({request}) => {
  const response = await request.get('https://reqres.in/api/users?page=2');
  var responseJson = await response.json();
  console.log(responseJson);
  expect(response.status()).toBe(201);
  expect(responseJson.data[0].first_name).toBe('Michael');
})
```

## POST Method

- **Purpose:** Send data to the server to **create or update** something.
- **Example Use Case:** Submitting a form, uploading a file, logging in.
- **Characteristics:**
  - Data is sent in the **body** of the request (not visible in URL).
  - **More secure** for sensitive data.
  - Cannot be bookmarked or cached.
  - Typically used for **write** operations.

## PUT Method

- **Purpose:** Update or replace existing data on the server.
- **Example Use Case:** Updating a user's profile, changing a product's details.
- **Characteristics:**
  - Sends data in the **request body**.

- **Idempotent:** Sending the same request multiple times results in the same outcome.
- Replaces the entire resource (unless it's a partial update, which is usually done with PATCH).

## DELETE Method

- **Purpose:** Remove data from the server.
- **Example Use Case:** Deleting a user account, removing a post.
- **Characteristics:**
  - Usually doesn't require a body (but can include one).
  - **Idempotent:** Deleting the same resource multiple times has the same effect.

**Example:**

```
apiTest.spec.ts > test('Get user Details using GET API' callback
import { test, expect } from '@playwright/test';

test('Get user Details using GET API', async ({ request }) => {
  const response = await request.get('https://reqres.in/api/users?page=2');
  var responseJson = await response.json();
  console.log(responseJson);
  expect(response.status()).toBe(201);
  expect(responseJson.data[0].first_name).toBe('Michael');
})

const response = await request.post('https://reqres.in/api/users', {
  data: user,
  headers: { "ACCEPT": "applications/JSON" }
});
var responseJson = await response.json();
expect(response.status()).toBe(201);
expect(responseJson.name).toBe(`${user.name}`);
expect(responseJson.job).toBe(`${user.job}`);
userId = responseJson.id;
})
```



apitest.spec.ts > test(Update User using PUT API) callback > response

```
test('Update User using PUT API', async ({ request }) => {
  var user = {
    "name": "automation",
    "job": "course"
  }

  const response = await request.put('https://reqres.in/api/users/' +
    data:user,
    headers:{"ACCEPT":"applications/JSON"}
  );
  var responseJson = await response.json();
}
```

apitest.spec.ts > test(Update User using PUT API) callback > user

```
5
6 const response = await request.put('https://reqres.in/api/users/' + userId, {
7   data:user,
8   headers:{"ACCEPT":"applications/JSON"}
9 });
10 var responseJson = await response.json();
11 expect(response.status()).toBe(201);
12 expect(responseJson.name).toBe(`${user.name}`);
13 expect(responseJson.job).toBe(`${user.job}`);
14 userId = responseJson.id;
15
```

```
    expect(responseJson.job).toBe(`${user.job}`);
    userId = responseJson.id;
  });

  test('Delete User using DELETE API', async ({ request }) => {
    const response = await request.delete('https://reqres.in/api/users/' + userId);
    expect(response.status()).toBe(204);

    const response2 = await request.get('https://reqres.in/api/users/' + userId);
    console.log(await response2.json());
  });
}
```

## **BDD- Behaviour-Driven Deployment:**

“Business-Driven Development means we first understand what the business wants to achieve, then we build software that supports those goals. It helps make sure IT and business are working together, and the final product is useful and valuable.”

**Cucumber:** It is a tool which allows the user to write the test in a business oriented manner.

Gherkin is the syntax we need to follow in order to write test in cucumber.

### **Benefits of cucumber:**

Even the business stakeholders can easily understand the automation testing.

Readability promotes teamwork

Gherkin is easy to adopt

### **Installation & Setup**

#### **Procedure**

- Create a new folder and follow the playwright installation steps.
- Download the following dependencies:
  - **@cucumber/cucumber:** To download the actual cucumber software.

**Syntax:** npm i -D @cucumber/cucumber

- **ts-node:** This is a TypeScript execution and REPL for NodeJS with source map and native support.

**Syntax:** npm i -D ts-node

- Install the cucumber extension from the VSCode extensions.





- This extension adds supports for using cucumber in the Visual Studio Code.
- This extension is powered by the cucumber language server.
- **Advantages of this extension:**
  - **Give the autocompletion of Gherkin steps** - Whenever any Gherkin steps are written, on entering just 1 or 2 words, it will give an auto completion.
  - **Other Advantages:**



- Delete the 'tests' and 'tests-examples' folders created during installation of playwright.
- Create a folder 'src'
- Inside the 'src' folder, create the following folders:
  - tests - Inside this 2 other folders:
    - features - Contains feature files which are written in the Gherkin implementation.
    - steps - Contains playwright files, which will hold the playwright code, which will help to execute the tests written as part of the feature file.
- Create a configuration file for cucumber - cucumber.json - In which all the configurations are written in the form of objects.

```
{
  "default":{
    "paths":["src/test/features"],
    "require":["src/test/steps/*.ts"],
    "requireModule":["ts-node/register"]
  }
}
```

- "paths": Need to give where the feature files are located.
- "require": Need to give the path of playwright TypeScript code. /\*.ts to refer all the files
- "requireModule": Needed for internal conversion of ts to js
- In the package.json file:

```
"scripts": {
  "test":"cucumber-js test"
```

},

- To run cucumber in the form of JS, which is extracted using the ts-node internal conversion mentioned above.

### **Additional Configuration to be done (Executed successfully without this)**

With the previous configuration, we have installed Cucumber and completed needed configurations. But with the latest version of "@cucumber/cucumber" --> It will give error when executing the test. The error it will throw is

```
import { Given, When, Then } from "@cucumber/cucumber";
```

SyntaxError: Cannot use import statement outside a module

To fix this error we need to create tsconfig.json file in the root folder and add following code

```
{  
  "compilerOptions": {  
    "module": "commonjs",  
    "moduleResolution": "node"  
  }  
}
```

This will fix the error.

## Procedure to write Cucumber test

- Create a <filename>.feature file in the features folder.
- In the file write the whole testing flow in Gherkin language.
- In the cucumber.json file:

```
{  
  "default":{  
    "paths":["src/test/features"],  
    "formatOptions":{  
      "snippetInterface": "async-await"  
    },  
    "require":["src/test/steps/*.ts"],  
    "requireModule":["ts-node/register"]  
  }  
}
```

- In "formatOptions": the "snippetInterface": "async-await" is written to create async await using functions as playwright requires such functions.
- Run the .feature file - npm run tests
- It will give the step definitions and/or the format in which the test script needs to be written in.
- Copy paste the same in a <filename>.ts created in the steps folder created for the corresponding .feature file.
- Edit the code as required.
- For attaching the .ts file to the .feature:
  - Settings -> search for 'cucumber' -> click on edit in the .json -> add

```
"cucumber.glue":[
```

```

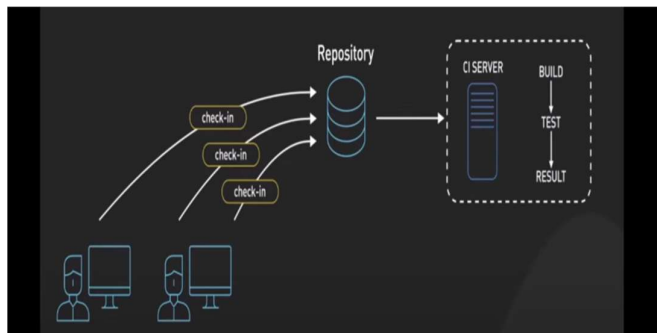
"*specs/**/*.cs",
"features/**/*.js",
"features/**/*.jsx",
"features/**/*.php",
"features/**/*.py",
"features/**/*.rs",
"features/**/*.rb",
"features/**/*.ts",
"features/**/*.tsx",
"features/**/*._test.go",
"src/test/**/*.java",
"tests/**/*.py",
"tests/**/*.rs",
"src/test/steps/*.ts"

```

]

- "src/test/steps/\*.ts" -> needs to be added after adding the cucumber.glue value; rest came automatically on adding the "cucumber.glue" key.
- After editing the code, to execute the code, again in the terminal type - npm run test.

### Continuous Integration -Continuous Delivery/Deployment:



## **What is Continuous Integration**

Continuous integration (CI) is a practice of integrating code from multiple developers into a central repository (or branch), multiple times per day.

A CI build consists of automated processes that:

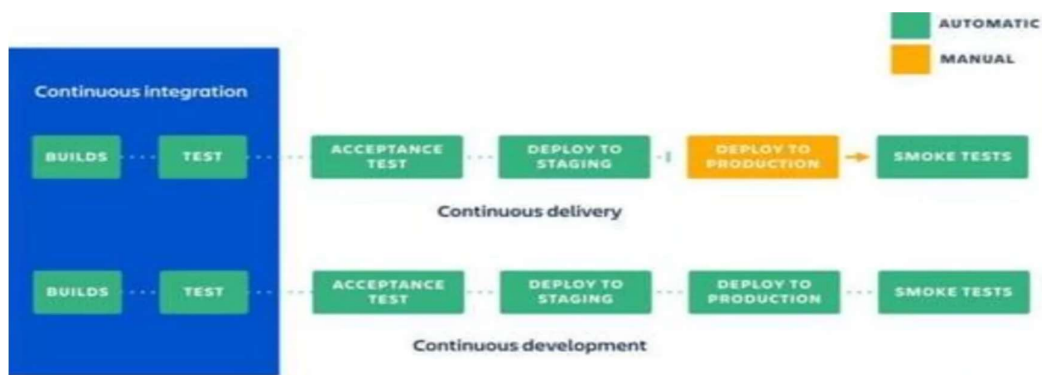
- Run automatic code quality scan, and generate a report how well latest code changes adhere to good coding practices
- Build code and run any automated tests (mostly unit tests) that are written to make sure the changes don't break any functionality
- Generate and publish a test coverage report

## **What is Continuous Deployment**

- Continuous deployment goes one step further than continuous delivery.
- Every change that passed all stages of production pipeline is released to customers without any human intervention
- Only a failed step can prevent a new change to be deployed to production.

## Benefits of CICD

- Higher Quality
  - Less bugs shipped to Productions as bugs are captured early by automated tests
  - Developers are alerted as soon as they break a build and can work on fixing it before moving to another task
- Low Risk Releases
- Lower Cost - Fixing Bug in Dev is less Cost than fixing in PROD
- Time to Market faster



## Challenges when implementing CICD

- Require organization changes and mindset shifts
- Deep technical knowledge to automate the entire process
- Team is required to use various technologies depending on application stack
- Infrastructure cost is a potential issue since team requires at least two identical environments (staging and production) that allows the team tests new changes without disrupting production.

